

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
ASSESSMENT TOOL 2 – MODEL COMPARISON PRESENTATION (BERT VS GPT VS T5 VS LLAMA)

Course Code:	CSA65	Course Title:	Generative AI and Large Language Models
CO Assessed:	CO4 – Precision (BL3, BL4,BL5)	Assessment:	Assessment Tool 2 - Model Comparison Presentation (BERT vs GPT vs T5 vs LLaMA)
Weightage:	10% of CIA	Total Marks:	2 * 50= 100
CO4 Statement:	<i>Design and implement multimodal Generative AI applications integrating text, image, and speech models for engineering applications.(BL3, BL4,BL5)</i>		
Student Name:	V Rohan	Reg. No.:	192472227
Section / Batch:	CSE AI	Date:	21 / 08 / 2026

Instructions to Students

- **Answer any 2 questions. Each question carries 50 marks. Total: 100 marks.**
- **Compare all four models: BERT, GPT, T5, and LLaMA.**
- Explain the **architecture and working principle** of each model.
- Compare their **pre-training objectives and capabilities**.
- Map each model to appropriate **real-world applications**.
- Identify the **strengths and limitations** of each model.
- Support the presentation using **credible research papers, official documentation, benchmarks, or other reliable sources**.
- Include at least one **comparison table or diagram**.
- Use relevant **real-world case studies or application examples**.
- Conclude by identifying the **most suitable model for the selected application** and justify the decision.
- Include proper **references/citations** in the presentation.

Code of Conduct

I ___ V Rohan (192472227) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: ___ Rohan V _____

SECTION A: MODEL COMPARISON PRESENTATION (BERT VS GPT VS T5 VS LLAMA)

Question 1

Compare the architectures of BERT, GPT, T5, and LLaMA. Explain the major architectural differences and identify the type of NLP tasks each architecture is best suited for.

1. Introduction

The Transformer architecture introduced by Vaswani et al. (2017) is the common ancestor of BERT, GPT, T5, and LLaMA, but each model keeps only part of the original encoder-decoder design and adapts it for a different purpose. BERT keeps the encoder half and drops the decoder, GPT keeps the decoder half and drops the encoder, T5 keeps both halves in their original encoder-decoder form, and LLaMA follows GPT's decoder-only lineage while modernising several of its internal components. These choices are not cosmetic: the presence or absence of an encoder, the direction in which self-attention is allowed to look, and the way positional information is injected together determine whether a model is naturally suited to understanding a fixed piece of text, generating open-ended text, converting one piece of text into another, or all three at once. This comparison walks through the architecture of each model individually, then draws out the differences that matter most in practice, and finally maps each architecture to the NLP tasks it is best suited for.

2. Architectural Recap Table

Model	Architecture Type	Core Building Block	Attention Pattern	Typical Scale
BERT	Encoder-only Transformer	Bidirectional self-attention encoder stack	Fully bidirectional (every token attends to every other token)	110M (base) to 340M (large) parameters
GPT	Decoder-only Transformer	Masked (causal) self-attention decoder stack	Causal / left-to-right (a token attends only to earlier tokens)	117M (GPT-1) to 175B (GPT-3) parameters
T5	Encoder-decoder Transformer	Bidirectional encoder + causal decoder joined by cross-attention	Bidirectional in the encoder, causal with cross-attention in the decoder	60M (small) to 11B (XXL) parameters
LLaMA	Decoder-only Transformer (modernised)	Causal decoder stack with RMSNorm,	Causal / left-to-right, same family as GPT	7B to 65B parameters (LLaMA-1)

Model	Architecture Type	Core Building Block	Attention Pattern	Typical Scale
		SwiGLU, and rotary positional embeddings		

This table already hints at the central architectural split that runs through the rest of the comparison: BERT is encoder-only and bidirectional, GPT and LLaMA are decoder-only and causal, and T5 is the only one of the four that keeps a genuine encoder-decoder pair.

3. BERT: Architecture and Working Principle

BERT (Bidirectional Encoder Representations from Transformers) is built entirely from the encoder side of the original Transformer. Each encoder layer applies multi-head self-attention followed by a position-wise feed-forward network, with residual connections and layer normalisation around both sub-layers. Critically, BERT's self-attention is not masked in any direction, so when the model computes a representation for a given token it can look at every other token in the sequence, both to its left and to its right, at the same time. This is what "bidirectional" means in BERT's name, and it is why BERT produces very strong contextual representations of a fixed input: a word's representation is informed by the entire sentence around it rather than only the words that came before it. BERT has no decoder and therefore no built-in mechanism for generating new tokens one at a time; to use it for a downstream task, a small task-specific output layer (a classification head, a span-prediction head for question answering, and so on) is attached on top of the encoder's final hidden states and trained together with the rest of the network. BERT's input representation combines WordPiece token embeddings, learned absolute positional embeddings, and segment embeddings that mark whether a token belongs to the first or second sentence in a pair, which supports its next-sentence-prediction pre-training task.

4. GPT: Architecture and Working Principle

GPT (Generative Pre-trained Transformer) is built entirely from the decoder side of the original Transformer, but without the cross-attention sub-layer that a full decoder would use to attend to an encoder's output, since there is no encoder present. Each GPT layer applies masked multi-head self-attention followed by a feed-forward network, again with residual connections and layer normalisation. The mask is the defining feature: it forces every token's attention to look only at itself and the tokens before it, never at tokens that come later in the sequence. This causal masking is what allows GPT to be trained simply by predicting the next token at every position and, at inference time, to generate text one token at a time by repeatedly feeding its own previous output back in as input. Because generation and pre-training use the same left-to-right mechanism, GPT does not need a separate output head bolted on for generation the way BERT does for classification; the same language-modelling head that was pre-trained is the one used to produce new text. GPT's positional information in the original design comes from learned absolute position embeddings added to the token embeddings before the first layer.

5. T5: Architecture and Working Principle

T5 (Text-to-Text Transfer Transformer) keeps the complete encoder-decoder structure of the original Transformer. The encoder is bidirectional, exactly like BERT's, and processes the entire input sequence at once to build a contextual representation of it. The decoder is causal, exactly like GPT's, but it also contains a cross-attention sub-layer in every layer that lets each decoder position attend over the encoder's output representations. This means T5 generates its output autoregressively, one token at a time, while continuously consulting a full bidirectional understanding of the input through cross-attention — a capability that neither a pure encoder-only nor a pure decoder-only model has on its own. T5's key design decision is to frame every task, whether it is originally a classification task, a translation task, or a summarisation task, as converting an input text string into an output text string, with a short task-prefix (such as "summarize:" or "translate English to German:") prepended to the input so a single model and a single architecture can be reused across many tasks. T5 also replaces BERT's and GPT's absolute positional embeddings with a simpler relative-position bias added directly inside the attention score computation.

6. LLaMA: Architecture and Working Principle

LLaMA (Large Language Model Meta AI) is architecturally a decoder-only Transformer in the same family as GPT, but Touvron et al. (2023) modernise several internal components based on techniques that were shown to improve training stability and efficiency after GPT-3 was released. LLaMA normalises the input of each sub-layer rather than the output (pre-normalisation) and replaces standard LayerNorm with RMSNorm, which is computationally cheaper. It replaces the ReLU activation used in earlier Transformer feed-forward blocks with a SwiGLU activation, which was found to improve quality for a fixed compute budget. Most significantly, LLaMA replaces learned absolute positional embeddings with rotary positional embeddings (RoPE), which encode position by rotating the query and key vectors as a function of their position rather than adding a separate position vector to the token embedding; this generalises better to sequence lengths not seen during training. Despite these internal changes, the outer architecture is still a stack of causal decoder layers trained with next-token prediction, so LLaMA inherits GPT's core behaviour: it processes a prompt left-to-right and continues it, which makes it naturally suited to the same category of tasks as GPT while being more parameter-efficient because of these architectural refinements and its training recipe.

7. Major Architectural Differences

Four dimensions capture most of what separates the four models. The first is the presence and direction of the encoder: BERT and T5 both contain a bidirectional encoder that can see the whole input at once, while GPT and LLaMA have no encoder at all and only ever see text causally. The second is the presence of a decoder: GPT, T5, and LLaMA all contain a component that generates output autoregressively, while BERT has no decoder and therefore cannot natively generate free-form text. The third dimension is attention masking: BERT's attention is fully open, GPT's and LLaMA's attention is strictly causal, and T5 uses both — open attention inside its encoder and causal attention with an added cross-attention link inside its decoder. The fourth dimension is positional encoding: BERT and the original GPT use learned absolute

position embeddings, T5 uses a relative-position bias inside attention, and LLaMA uses rotary position embeddings; this progression reflects a broader trend in Transformer design toward positional schemes that generalise better to long or variable-length sequences. Taken together, these differences explain why BERT excels at building a rich representation of a fixed piece of text but cannot generate one, why GPT and LLaMA excel at open-ended generation but only ever condition on what came before, and why T5 alone can combine full bidirectional understanding of an input with autoregressive generation of a differently-shaped output.

8. Task-Suitability Mapping

Model	Architecture-Driven Strength	Best-Fit NLP Tasks
BERT	Deep bidirectional understanding of a fixed input; no native generation	Text classification, sentiment analysis, named entity recognition, extractive question answering, sentence-pair tasks
GPT	Strong autoregressive generation conditioned on preceding context	Open-ended text generation, dialogue, creative writing, few-shot in-context tasks, code generation
T5	Bidirectional understanding combined with controlled generation through a unified text-to-text interface	Abstractive summarisation, machine translation, generative question answering, any task that can be phrased as text-in/text-out
LLaMA	GPT-style generation with better efficiency per parameter and stronger long-context handling via RoPE	General-purpose assistants, instruction following after fine-tuning, reasoning, retrieval-augmented generation, domain-specific chat applications

The pattern that emerges is architectural rather than incidental: whenever a task ends with picking a label or a span from a fixed input, an encoder-only model such as BERT is the natural fit; whenever a task ends with producing new free-form text, a decoder-containing model — GPT, T5, or LLaMA — is required, with T5 preferred when the transformation from input to output is well-defined and structured, and GPT or LLaMA preferred when the generation is more open-ended or needs to be steered through a prompt rather than through training on a fixed input-output format.

9. Real-World Illustration

Consider a legal-tech company building three separate features on top of a contracts database: (1) flagging whether a clause is a "termination clause" or not, (2) producing a one-paragraph plain-language summary of a ten-page contract, and (3) letting a lawyer ask free-form questions about a specific document. The clause-flagging feature is a classification problem over a fixed span of text, so a fine-tuned BERT-style

encoder is architecturally the right fit — it is small, fast, and its bidirectional representation is exactly what a classification head needs. The summarisation feature requires transforming a long input into a shorter, differently-worded output, which is precisely the text-to-text pattern T5 was built for, making a fine-tuned T5 (or an instruction-tuned variant) a strong choice. The open-ended question-answering feature benefits from a decoder-only model such as GPT or LLaMA that can be prompted with the document as context and asked an arbitrary question, generating a free-form answer without needing a separate model trained for every possible question type. This single company scenario shows how the architectural differences translate directly into which model a team would reach for on each sub-problem, rather than picking one model and forcing every task through it.

10. Strengths and Limitations by Architecture

Model	Architectural Strength	Architectural Limitation
BERT	Best-in-class contextual understanding of fixed text; small and fast at inference	Cannot generate free-form text natively; needs a trained head for every new task
GPT	Simple, uniform architecture that supports both understanding and generation through prompting	Only sees prior context (no bidirectional view), so tasks needing a full view of both sides of a token are weaker than BERT's
T5	Only architecture with a true bidirectional encoder feeding an autoregressive decoder; one interface for many tasks	Larger and slower per parameter than a decoder-only model, since it runs both an encoder and a decoder
LLaMA	Efficient decoder design (RMSNorm, SwiGLU, RoPE) gives strong performance per parameter and good long-context behaviour	Still causal-only, so — like GPT — it lacks BERT's fully bidirectional view of an input for pure understanding tasks

Question 25

Compare BERT, GPT, T5, and LLaMA for zero-shot, one-shot, and few-shot learning. Explain how prompting and fine-tuning influence their performance.

1. Introduction

Zero-shot, one-shot, and few-shot learning describe how much task-specific guidance a language model needs before it can perform a new job correctly. A model is said to work in a zero-shot setting when it is given only an instruction and must complete the task without seeing any worked examples. One-shot learning allows the model a single demonstration, while few-shot learning provides a handful of examples — typically two to a few dozen — inside the input itself. These settings matter because they decide how flexible a deployed system can be: a model that performs well zero-shot can be pointed at a brand-new task in production without retraining, whereas a model that only performs well after fine-tuning needs labelled data and a training pipeline for every new use case.

BERT, GPT, T5, and LLaMA sit at different points on this spectrum because of how they were built. BERT is an encoder-only model trained with masked-language modelling; it was designed to be fine-tuned on labelled data rather than prompted, so it was never built to be a strong few-shot learner in the modern sense. GPT is a decoder-only, autoregressive model trained on next-token prediction; from GPT-2 onward it was explicitly studied for zero-shot and few-shot behaviour, and GPT-3 (Brown et al., 2020) is the paper that popularised in-context few-shot learning as a practical alternative to fine-tuning. T5 reframes every NLP task as text-to-text generation and was trained with a mixture of unsupervised span-corruption and supervised tasks, giving it a middle position between BERT-style fine-tuning and GPT-style prompting. LLaMA is a modern decoder-only foundation model trained on trillions of tokens; its scale and training recipe give it strong zero-shot and few-shot ability similar in spirit to GPT-3, while remaining open-weight and easy to fine-tune. The sections below compare all four models across the three learning regimes and then explain how prompting and fine-tuning each shape their real-world performance.

2. Quick Architectural Recap

Because a model's ability to learn from few or zero examples is tied directly to its architecture and pre-training objective, a short recap is useful before comparing the learning regimes.

Model	Architecture	Pre-training Objective	Primary Orientation
BERT	Encoder-only Transformer	Masked Language Modelling + Next Sentence Prediction	Language understanding (classification, extraction)
GPT	Decoder-only Transformer	Causal (autoregressive) Language Modelling	Language generation, in-context learning

Model	Architecture	Pre-training Objective	Primary Orientation
T5	Encoder-Decoder Transformer	Text-to-Text span corruption on a unified format	Understanding and generation via one interface
LLaMA	Decoder-only Transformer (large-scale)	Causal Language Modelling on massive curated corpora	General-purpose foundation model for prompting and fine-tuning

This recap shows the underlying reason the four models diverge sharply once no labelled task data is available: only the decoder-only, autoregressive models (GPT and LLaMA) were trained in a way that naturally supports predicting a continuation of a prompt, which is exactly the mechanism that in-context learning relies on.

3. Comparison in the Zero-Shot Setting

Zero-shot learning means the model receives only a natural-language instruction — for example "Classify the sentiment of this review" — with no worked examples at all.

3.1 BERT. BERT was pre-trained to fill in masked tokens and to predict sentence relationships, not to follow instructions. It has no native mechanism for reading a task description and acting on it, because its output layer for any downstream task (classification, span extraction) has to be attached and trained afterward. As a result BERT is a poor zero-shot learner on its own; the only zero-shot-like behaviour that BERT variants show is when the masked-language-modelling head itself is repurposed — for instance filling a cloze-style prompt such as "This movie was [MASK]" to guess sentiment words — and even this depends heavily on prompt design and performs far below GPT-style zero-shot prompting.

3.2 GPT. GPT's causal-language-modelling objective naturally supports zero-shot use: because the model always predicts the next token given everything before it, an instruction can simply be written as the start of the sequence and the model will continue it in a way that often answers the instruction. GPT-2 already showed measurable zero-shot performance on tasks such as reading comprehension and summarisation, and GPT-3 substantially improved on this because of its scale, showing competitive zero-shot accuracy on a wide range of benchmarks without any parameter updates.

3.3 T5. T5 was trained on a large mixture of supervised NLP tasks phrased as text-to-text conversion (for example prefixing an input with "translate English to German:"), so it can generalise moderately well to zero-shot instructions that resemble its training task prefixes. Its zero-shot ability is weaker than GPT-3's on genuinely unseen task formats, because T5's training data was constructed around a fixed, known set of task templates rather than open-ended natural-language instructions.

3.4 LLaMA. LLaMA follows the GPT lineage architecturally, so it inherits the same natural affinity for zero-shot prompting. Touvron et al. (2023) report that LLaMA achieves zero-shot results competitive

with, and in several benchmarks better than, much larger closed models, largely because of the quality and volume of its pre-training corpus rather than any special zero-shot mechanism. Instruction-tuned derivatives of LLaMA (chat/instruct variants) push zero-shot performance further by aligning the base model's completions with instruction-following behaviour.

4. Comparison in the One-Shot Setting

One-shot learning adds exactly one demonstration to the prompt before the actual query, so the model has a single concrete example of the desired input-output mapping.

BERT again struggles here because there is no standard way to feed a demonstration into its objective; any "example" has to be turned into additional training signal through gradient updates rather than being read as context, so BERT effectively treats one-shot the same as zero-shot unless it is fine-tuned. GPT benefits noticeably from a single example: Brown et al. (2020) show a consistent jump in accuracy from zero-shot to one-shot on tasks such as translation and arithmetic, because the one example helps the model infer the expected output format even when it already understands the underlying task. T5 shows a smaller but real improvement in one-shot settings when the demonstration matches a format close to its pre-training task mixture, but it is less able to generalise the demonstration to genuinely novel formats. LLaMA behaves like GPT: a single example measurably improves output formatting and task alignment, and because LLaMA was trained on more tokens with cleaner data curation than the original GPT-3 corpus, its one-shot performance per parameter tends to be strong — the 13-billion-parameter LLaMA model was reported to outperform the much larger 175-billion-parameter GPT-3 on several one-shot and few-shot benchmarks.

5. Comparison in the Few-Shot Setting

Few-shot learning gives the model several demonstrations (commonly 4 to 32) inside the prompt, letting it infer the task pattern purely from context, a behaviour known as in-context learning.

5.1 BERT. Because BERT has no autoregressive generation mechanism and was never trained with long instructive prompts, it cannot perform in-context few-shot learning in the GPT/LLaMA sense. When people describe "few-shot BERT," they usually mean few-shot fine-tuning: updating BERT's weights using only a handful of labelled examples, which is a different mechanism from in-context learning and still requires a training step, however small.

5.2 GPT. Few-shot in-context learning is the capability GPT-3 is best known for. With roughly 10 to 100 examples placed directly in the prompt and no gradient updates at all, GPT-3 approaches or matches fine-tuned state-of-the-art models on several benchmarks. Performance scales with model size: the effect is weak in small GPT models and becomes strong only once the model reaches tens of billions of parameters, which is why this ability is described as an emergent property of scale.

5.3 T5. T5's original design emphasised fine-tuning rather than in-context learning, so the base T5 models show weaker few-shot prompting ability than GPT-3 or LLaMA at comparable size. Later T5-family variants (such as instruction-tuned versions like FLAN-T5) close much of this gap by training explicitly

on a large number of tasks phrased as instructions, which improves both zero-shot and few-shot behaviour without changing the underlying architecture.

5.4 LLaMA. LLaMA shows strong few-shot in-context learning for the same architectural reasons as GPT, and its larger training-data volume relative to parameter count (following the Chinchilla-style scaling insight that models should be trained on far more tokens than earlier GPT-style models) gives it competitive or superior few-shot accuracy per parameter. This is a key practical advantage: LLaMA-13B was shown to match or beat GPT-3 (175B) on many few-shot tasks while being small enough to run on a single high-end GPU.

6. Summary Comparison Table

Learning Setting	BERT	GPT	T5	LLaMA
Zero-shot	Weak — needs cloze-style workarounds	Good, improves strongly with scale	Moderate, tied to known task prefixes	Strong, competitive with larger models
One-shot	Not natively supported	Clear improvement over zero-shot	Small improvement, format-dependent	Strong, efficient per parameter
Few-shot (in-context)	Not supported; needs fine-tuning instead	Signature strength (GPT-3)	Weaker unless instruction-tuned (FLAN-T5)	Strong, matches larger GPT models
Best-fit learning mode	Supervised fine-tuning	Prompting / in-context learning	Fine-tuning, or instruction-tuned prompting	Both prompting and efficient fine-tuning

7. How Prompting Influences Performance

Prompting is the practice of shaping the natural-language input so that a model's next-token or fill-in-the-blank prediction lines up with the desired task output, and its effectiveness depends heavily on the model's pre-training objective.

- BERT: prompting only works through masked-token cloze tricks and is fragile; small changes in wording cause large accuracy swings, so BERT is rarely used with prompting in production.
- GPT: prompting is the primary interface. Careful instruction wording, example ordering, and formatting (prompt engineering) can change accuracy by tens of percentage points without touching the model's weights, which is why techniques such as chain-of-thought prompting were discovered on GPT-family models.
- T5: responds well to prompts that resemble its training-time task prefixes (e.g., "summarize:", "translate English to French:"); prompting for tasks outside this template set is less reliable unless the model has been instruction-tuned.

- LLaMA: responds to prompting similarly to GPT, and instruction-tuned/chat-aligned LLaMA variants are noticeably more prompt-friendly than the raw base model, showing that prompting effectiveness is partly a property of the base pre-training and partly a property of any later alignment step.

Overall, prompting is most powerful for decoder-only, autoregressive models (GPT, LLaMA) because their training objective already rewards continuing a plausible sequence of text, and an instruction is simply another sequence to continue. It is least powerful for BERT because the model was never trained to condition its output on an open-ended instruction.

8. How Fine-Tuning Influences Performance

Fine-tuning updates a model's parameters using labelled examples for a specific task, and its impact also differs sharply across the four models.

- BERT: fine-tuning is essential rather than optional — the base model is only useful once a task-specific head is attached and trained, and even a few hundred labelled examples typically produce large accuracy gains because BERT's contextual representations transfer well.
- GPT: fine-tuning further boosts accuracy over prompting alone, especially for narrow, repetitive tasks, but is often skipped in practice because few-shot prompting already gets close to fine-tuned performance at much lower cost; when fine-tuning is used, parameter-efficient methods (e.g., LoRA, adapters) are common so that only a small fraction of weights are updated.
- T5: fine-tuning was the model's intended mode of use from the start, since the original T5 paper studied fine-tuning across many tasks systematically; it remains one of the strongest fine-tuning-based performers per parameter, particularly for structured generation tasks like summarisation and translation.
- LLaMA: being open-weight, LLaMA is widely fine-tuned by the community and industry, from full fine-tuning to parameter-efficient methods (LoRA, QLoRA) and instruction/alignment tuning; this flexibility, combined with already-strong zero/few-shot ability, is a major reason LLaMA-based models are popular for building custom domain-specific assistants.

In general, fine-tuning narrows the performance gap between all four models on a specific task, because it directly optimises the model for that task's data distribution. The trade-off is that fine-tuning requires labelled data, compute, and a maintenance pipeline for every new task, while prompting a strong zero/few-shot model like GPT or LLaMA avoids that cost at the price of slightly lower peak accuracy on narrow, well-defined tasks.

A useful way to see the relationship between prompting and fine-tuning is as two ends of a data-efficiency spectrum rather than two unrelated techniques. At one end, zero-shot prompting uses no task-specific data at all and relies entirely on what the model learned during pre-training. Moving along the spectrum, few-shot prompting uses a handful of examples placed in context without changing any weights. Parameter-efficient fine-tuning methods such as LoRA sit further along the spectrum: they use hundreds to thousands of labelled examples and update only a small number of additional parameters, which keeps training cheap

while still adapting the model's behaviour. Full fine-tuning sits at the far end, updating every parameter using the largest available labelled dataset for maximum task-specific accuracy. BERT and T5 were designed to operate mainly at the fine-tuning end of this spectrum, GPT and LLaMA were designed to operate well across the entire spectrum, and this difference — more than raw parameter count — is what explains their contrasting zero-shot and few-shot behaviour.

9. Real-World Illustration

Consider a college helpdesk that wants to auto-tag incoming student queries into categories such as "fees," "exam schedule," and "hostel." Using BERT for this task would mean collecting a labelled dataset of past queries and fine-tuning a classification head — an approach that gives high accuracy once enough labelled data exists but cannot handle a brand-new category without retraining. Using GPT or LLaMA, the same task can be handled zero-shot or few-shot by describing the categories in the prompt and, optionally, giving two or three example queries; this lets the helpdesk add a new category by editing the prompt instead of retraining a model. If the helpdesk later needs very high accuracy and has thousands of labelled tickets, fine-tuning a smaller open model such as LLaMA with LoRA — or fine-tuning T5 to generate the category as text — becomes the more cost-effective long-term solution. This example shows why organisations often combine both regimes: prompting for fast prototyping and flexibility, fine-tuning for stable, high-volume, well-defined tasks.

10. Limitations and Practical Considerations

Each model's position on the zero-shot to fine-tuning spectrum brings its own practical costs. BERT's dependence on fine-tuning means every new task requires a labelling effort and a training run, but the resulting models are small, fast, and cheap to run at inference time, which is why BERT remains common in latency-sensitive production classifiers. GPT's few-shot strength comes at the cost of longer prompts, since including several demonstrations in every request increases the number of tokens processed and therefore the inference cost and latency, and the largest GPT models are typically only accessible through a paid API rather than run locally. T5's reliance on task-prefix formatting means it can silently underperform on tasks that do not resemble its pre-training mixture unless the prompt is carefully engineered or the model is instruction-tuned first. LLaMA's openness is an advantage for customisation, but it also means the base model has no built-in safety alignment or instruction-following behaviour until it is explicitly fine-tuned or paired with an instruction-tuned checkpoint, so organisations adopting LLaMA directly must budget time for this alignment step. These trade-offs mean that the "best" model in the zero-shot, one-shot, and few-shot sense is not automatically the best choice for every deployment; cost, latency, data availability, and the need for customisation all influence which model and which learning regime is appropriate.

RUBRICS FOR CALCULATION

Criteria	Weight age	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Architectural Comparison Accuracy	30%	Accurate, in-depth comparison of architectures and use-cases	Accurate comparison with minor gaps	Superficial comparison of models	Inaccurate or confused comparison
Application Mapping	25%	Clearly maps each model to appropriate real-world applications	Reasonable mapping to applications	Weak mapping to applications	No meaningful mapping presented
Research Depth	20%	Well-researched with credible sources/benchmarks cited	Adequately researched	Limited research depth	Unresearched / opinion-based content
Delivery	15%	Confident, engaging, well-paced delivery	Clear delivery with minor pacing issues	Hesitant delivery	Unclear or unprepared delivery
Slide Design & References	10%	Well-designed slides with proper references	Adequate slides, minor reference gaps	Basic slides, poor referencing	No slides or references provided

Marks Evaluation:

Question No.	Architectural Comparison Accuracy (30%)	Application Mapping (25%)	Research Depth (20%)	Delivery (15%)	Slide Design & References (10%)	Total Marks (100)
Q1						
Q2						
Overall Total (100)						